

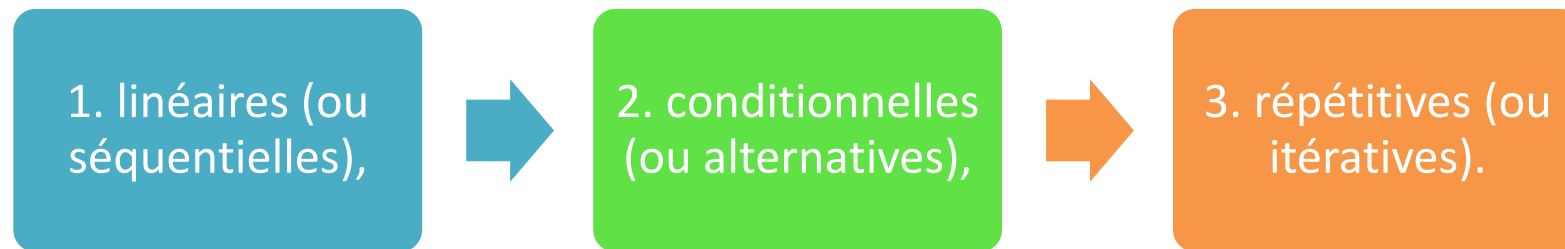
Chapitre 3 : Les différentes structures algorithmiques

Prof : Saidani Khaoula

Année universitaire : 2025/2026

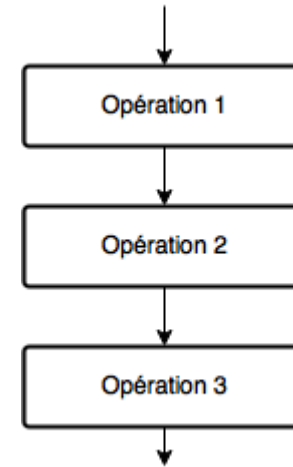
Introduction

- comment construire des algorithmes et utiliser des structures de données pour résoudre des problèmes en informatique, en se concentrant sur les concepts fondamentaux indépendamment d'un langage de programmation spécifique.
- Les structures algorithmiques se divisent principalement en trois catégories :

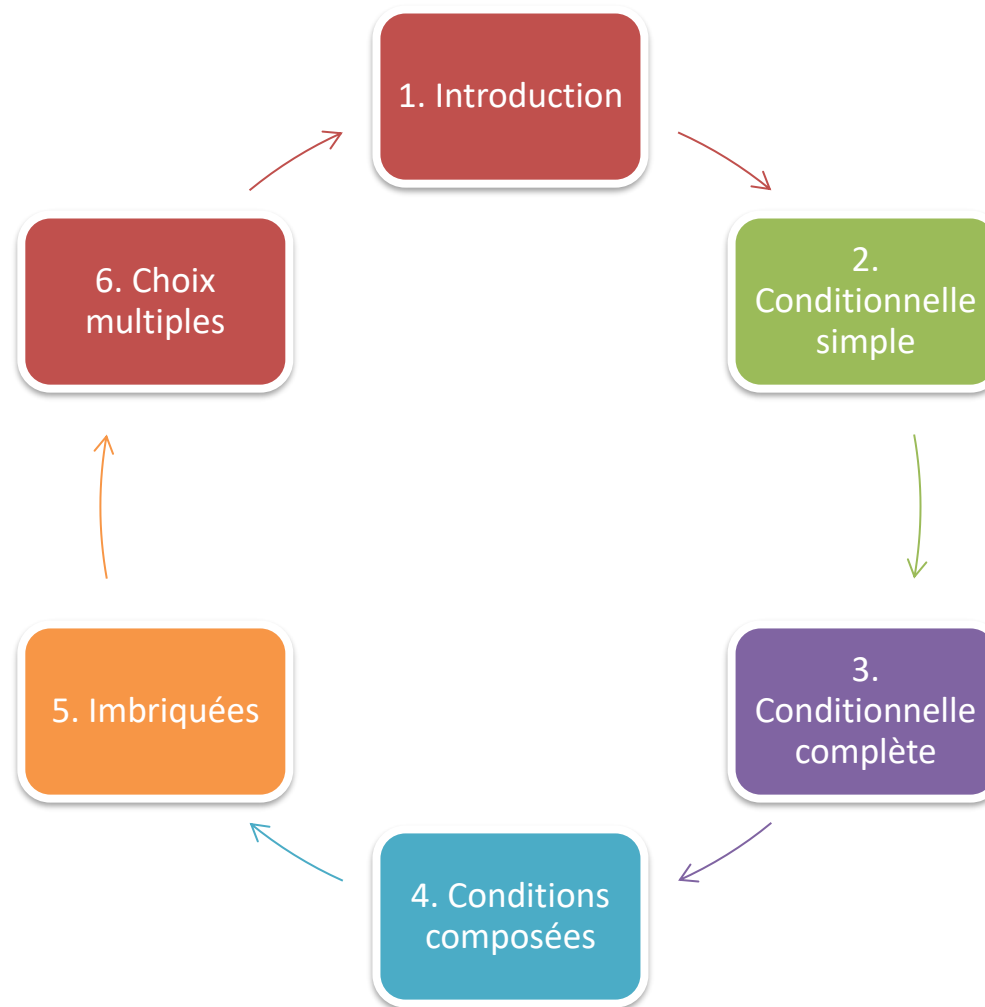


Structure Linéaire

- ✓ La structure linéaire est la structure de base de l'algorithmique.
- ✓ Elle se caractérise par une suite d'actions à exécuter successivement dans l'ordre de leur énoncé.

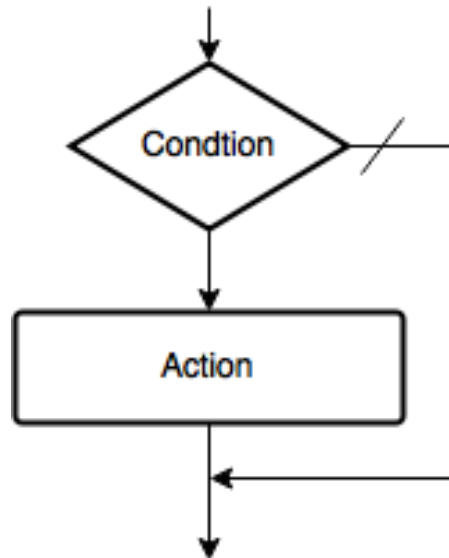


- Partie 1 : Structures Décisionnelles



Introduction

- Une structure conditionnelle permet de faire ou de ne pas faire une action.
- Exemples :



Algorithme

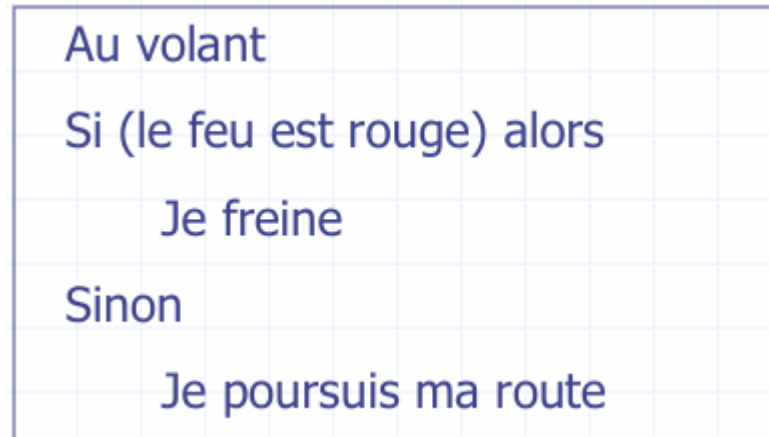
Si condition

Alors Action

Fin de si

Introduction

- Les algorithmes utilisent des décisions et des répétitions.
- Il arrive fréquemment dans la vie courante que notre conduite soit déterminée par un choix entre deux possibilités.
- Exemples :



```
graph TD; A[Au volant] --> B{Si (le feu est rouge) alors}; B --> C[Je freine]; B --> D{Sinon}; D --> E[Je poursuis ma route];
```

Au volant

Si (le feu est rouge) alors

Je freine

Sinon

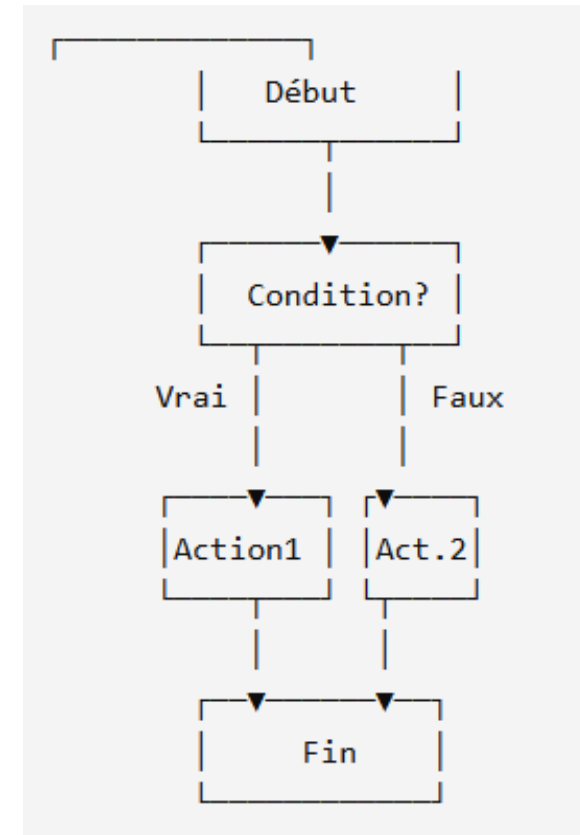
Je poursuis ma route

Introduction

- Cette situation est courante en algorithmique. L'instruction conditionnelle exprime cette possibilité de choix.
- Elle est liée à l'idée qu'une instruction ne s'exécute que si une condition C est vérifiée. Si la condition n'est pas vérifiée, c'est une autre instruction qui est exécutée

Structures Décisionnelles

- Une structure décisionnelle permet d'exécuter différentes actions selon une condition vraie ou fausse.



Structure Conditionnelle Simple

Si <condition> alors

 Action

Fin si

- Exemple :

Si (age >= 18) alors

 Écrire('Vous êtes majeur')

Fin si

Structure Conditionnelle Simple

- La condition doit être formulée de manière à n'avoir que deux réponses :
- ✓ **Oui** : la condition est vérifiée, exécuter action1.
- ✓ **Non** : la condition n'est pas vérifiée, exécuter Action2.
- Action1 et/ou Action2 peuvent être des séquences d'instructions des instructions conditionnelles, on parle alors d'instruction conditionnelle imbriquée.

Structure Conditionnelle Complete

- Exemple 1 :
Si <condition> alors
 Action1
Sinon
 Action2
Fin si
- Exemple 2 :
Si $(x > y)$ alors
 $\text{max} \leftarrow x$
Sinon
 $\text{max} \leftarrow y$
Fin si

Nature de la Condition

- La condition peut être simple ou complexe
 - L'expression d'une condition simple a l'une des formes suivantes :
 $A > B$, $A < B$, $A \geq B$, $A \leq B$, $A = B$ ou $A \neq B$ ou une variable booléenne
 - Une condition complexe combine plusieurs conditions simples à l'aide des opérations logiques ET, OU et NON.
 - Exemple : calculer l'expression $1/x + 1/y$

Structure Conditionnelle Composée

- Exemple : calculer l'expression $1/x + 1/y$

```
Algorithme somme_des_inverses
Var
x,y,s (réel)
Début
    Lire (x,y)
    Si (x<>0) ET (y<>0) alors
        s ← 1/x + 1/y
    écrire (s)
    Fin si
Fin
```

Conditions Simples

=	Égal à
<>	Différent de
>	Supérieur à
<	Inférieur à
>=	Supérieur ou égal
<=	Inférieur ou égal

Opérateurs Logiques

- ET (AND) : toutes les conditions doivent être vraies
- OU (OR) : au moins une condition est vraie

Conditions Composée

- Éditer la plus grande valeur. Le principe est le suivant : si la condition $x > y$ est vérifiée, x est le nombre cherché, sinon y est le nombre cherché.

```
Algorithme plus grand
Var
x,y,z (Réal)
Début
    Écrire("donner x")
    lire (x)
    écrire ("donner y")
    lire (y)
    si  $x > y$  alors
         $z \leftarrow x$ 
    sinon
         $z \leftarrow y$ 
    fin si
    écrire ("le plus grand de", x, "et de", y, "est", z)
Fin
```

Structure Conditionnelle imbriquée

- Si Action est une instruction conditionnelle alors on parle d'instruction conditionnelle imbriquée
- **Exemple:**
Un robot conduit une voiture , il peut exécuter 3 actions :
s'arrêter, ralentir ou passer en fonction de la couleur des feux (qui sera une variable saisie)

Structure Conditionnelle imbriquée

```
Algorithme circulation
Var
Couleur (chaîne de caractères)
Début
    écrire ("entrer la couleur")
    lire (Couleur)
    Si (Couleur = "Rouge") alors
        écrire ("S'arrêter")
    Sinon
        Si (Couleur = "Vert") alors
            écrire ("Passer")
        Sinon
            écrire ("Ralentir")
        Fin si
    Fin si
Fin Si
Fin
```

Structure Conditionnelle imbriquée

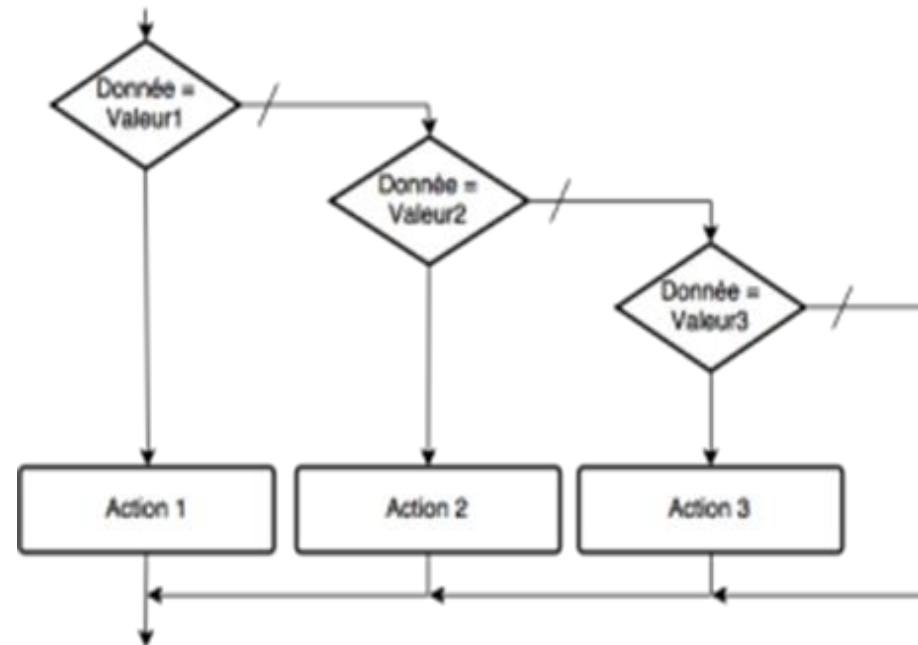
- Travail à faire:
- Calcul de remise à partir d'un montant lu en donnée, on détermine un montant net par application d'une remise de :
- 5% si le montant est compris entre 2000 DT (inclus) et 5000 DT (inclus).
- 10% si le montant est supérieur à 5000 DT.

Structure Conditionnelle à plusieurs Choix

- ✓ Lorsqu'une expression peut atteindre plusieurs valeurs possibles, et qu'il faut faire quelques choses de différent à chaque fois, on peut utiliser une structure conditionnelle à plusieurs choix (branchement multiple).
- ✓ Cette structure conditionnelle ressemble à la structure Si...Alors...Sinon, mais elle est plus efficace lorsqu'il s'agit de tester la même variable plusieurs fois.

Structure Alternative Multiple

➤ Exemple



Algorithme

Cas où Donnée vaut

Valeur 1 : Action 1

Valeur 2 : Action 2

Valeur 3 : Action 3

finCas

Structure Alternative Multiple

➤ Exemple

```
Suivant variable ou expression faire  
  <Liste_valeur_1> : instruction 1  
  <Liste_valeur_2> : instruction 2  
  ...  
  <Liste_valeur_n> : instruction n  
[Sinon : instruction par défaut]  
Fin Suivant
```

Structure Alternative Multiple

- ✓ L'expression ou valeur doit être une expression scalaire de même type que les valeurs dans les listes (liste_valeur_i).
- ✓ Si la valeur de l'expression appartient à l'une des listes de valeurs, l'instruction correspondante est exécutée et le branchement se termine (on ne regarde pas les autres valeurs).
- ✓ Si aucune de ces valeurs ne convient, l'instruction qui suit éventuellement SI NON (l'action par défaut) est exécutée.

Structure Alternative Multiple

➤ Exemple

Suivant nombre faire

0 : écrire ("Nombre nul")

1 : écrire ("Nombre égal à 1")

2,3,5,7 : écrire (" `Nombre premier")

4,6,8..10 : écrire (" `Nombre non premier")

Sinon : écrire ("Nombre négatif ou > 10")

Fin Suivant

Structure Alternative Multiple

- Un autre format pour les structures conditionnelles à plusieurs choix est le suivant:

```
Suivant variable ou expression faire
  <Condition_1> : instruction 1
  <Condition_2> : instruction 2
  ...
  <Condition_n> : instruction n
  [Sinon : instruction par défaut]
Fin Suivant
```

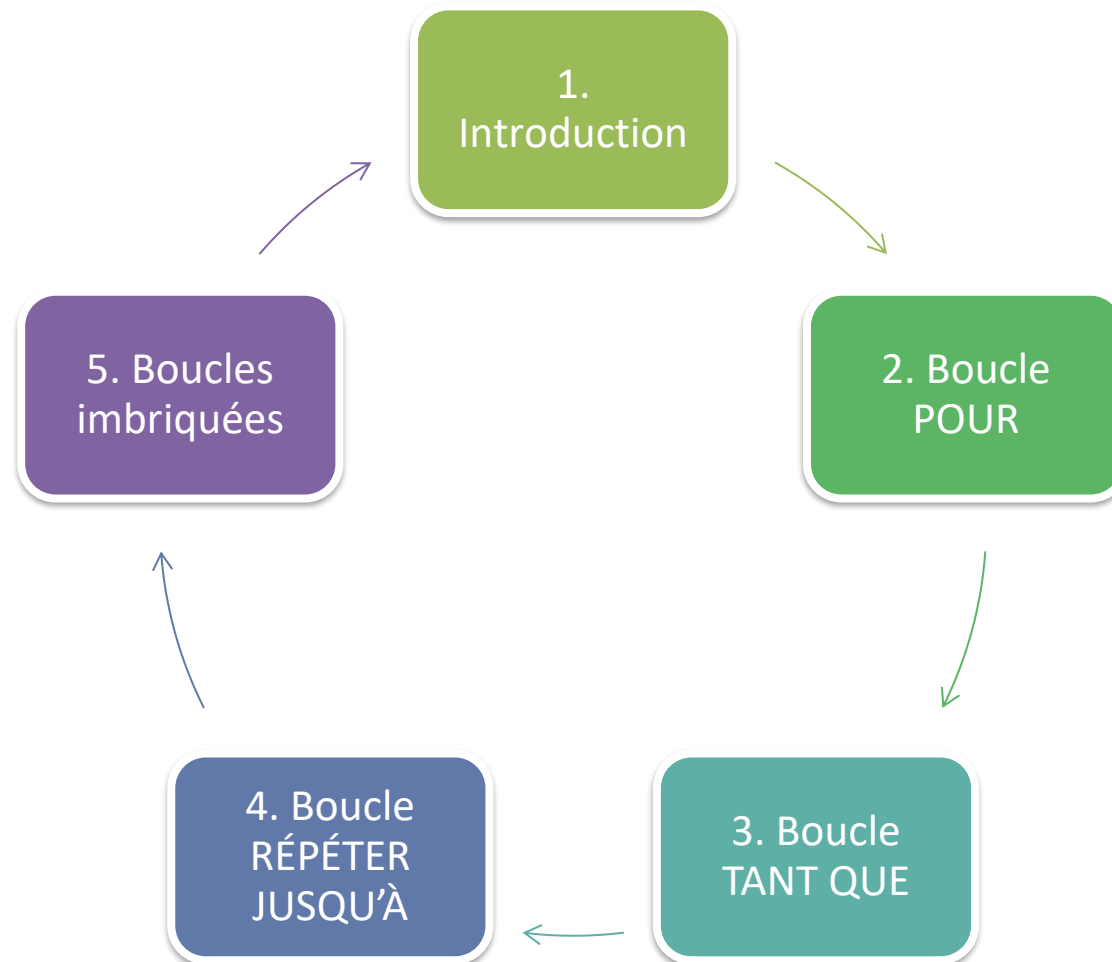
Structure Alternative Multiple

- Si la condition i est vérifiée, alors l'algorithme exécute l'instruction i ,
- Si aucune des conditions n'est vérifiée, l'instruction qui suit éventuellement SI NON (l'action par défaut) est exécutée.

Exemple

- **Travail à faire:** Écrire un algorithme qui permet de saisir une date sous la forme **jj mm aa** et d'afficher le nombre de jours du mois **mm** et le nombre de jours qui restent pour la fin de ce mois.

- Partie 2 : Structures Itératives



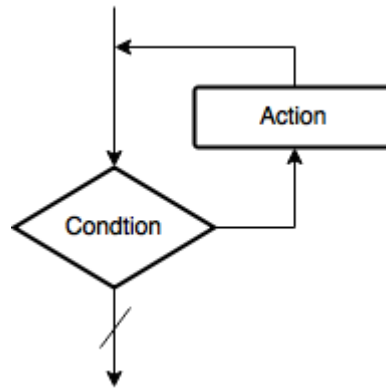
Introduction

- Les structures itératives, ou boucles, sont des outils fondamentaux en algorithmique pour répéter une série d'instructions plusieurs fois selon une condition.
- Les trois principales structures itératives sont
 - ✓ TantQue (où la condition est vérifiée avant chaque exécution du bloc d'instructions),
 - ✓ Répéter... Jusqu'à ce que (où le bloc d'instructions est exécuté au moins une fois avant la vérification de la condition) et,
 - ✓ Pour (utilisée lorsque le nombre d'itérations est connu à l'avance).

Structures Itératives

1. La boucle « Tant que ... »

- Cette structure permet de réaliser une action tant qu'une condition est vraie



Algorithme

Tant que Condition vraie

Action

Fin de tant que

Structures Itératives

La boucle « Tant que ... »

Tant que <variable> faire

 valeur1 : Action1

 valeur2 : Action2

Sinon : Action_par_défaut

Fin Tq

Exemple : Jours de la Semaine

Tanque numero_jour faire

1 : 'Lundi'

2 : 'Mardi'

...

7 : 'Dimanche'

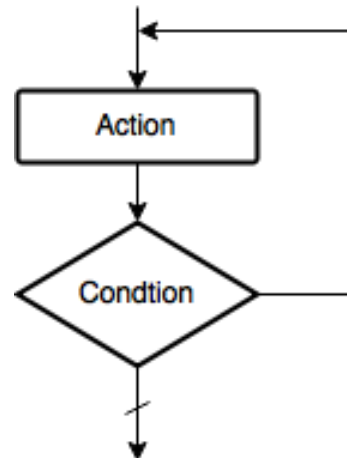
Sinon : 'Numéro invalide'

Fin Tq

Structures Itératives

2. La boucle « Répéter ... Jusqu'à »

- Exécute un traitement au moins une fois, puis vérifie une condition d'arrêt. La boucle continue jusqu'à ce que la condition devienne vraie.



Répéter
Instructions
Jusqu'à <condition>

Exemple

Algorithme somme_chiffres

Var

 nombre, somme, reste : Entier

Début

 Lire(nombre)

 somme $\leftarrow$ 0

 Répéter

 reste $\leftarrow$ nombre MOD 10

 somme $\leftarrow$ somme + reste

 nombre $\leftarrow$ nombre DIV 10

 Jusqu'à (nombre = 0)

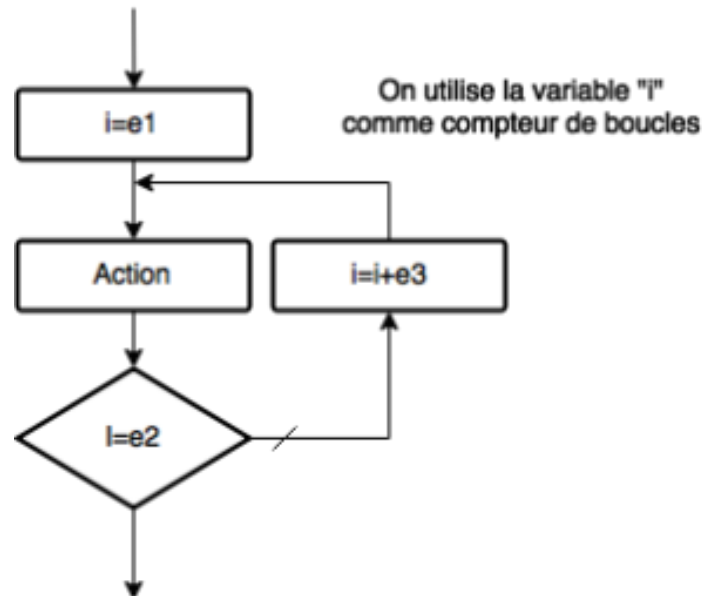
 Écrire("Somme : ", somme)

Fin

Structures Itératives

3. La boucle « Pour ... Faire »

- Idéale lorsque le nombre d'itérations est connu à l'avance. Elle répète un traitement un nombre de fois défini par une variable (par exemple, de 0 à n).



e1 : valeur initiale

e2 : valeur finale

e3 : pas

Algorithme

Pour (i de e1 à e3 par pas de e3) faire

Action

fin_pour

Exemple : Table de Multiplication

Pour i de 1 à 10 faire

Écrire(nombre, 'x', i, '=', nombre*i)

Fin Pour

Exemple : Factorielle

fact $\leftarrow$ 1

Pour i de 1 à n faire

 fact $\leftarrow$ fact * i

Fin Pour

Écrire(n, '! =', fact)

Conclusion

- Les structures décisionnelles permettent le choix d'actions.
- Les structures itératives permettent les répétitions.
- Elles rendent les algorithmes dynamiques et puissants.